

A Project Report
on
Movie Recommendation and Preference Mining System Using Data Mining Techniques
Semester - V

Submitted by

Gohel Ridham – 240170107041
Prajapati Vaidik Shaileshbhai – 240170107116

COMPUTER ENGINEERING DEPARTMENT
VISHWAKARMA GOVERNMENT ENGINEERING COLLEGE
CHANDKHEDA



Submitted to:
Niyati Shah
VGEC, Chandkheda



Gujarat Technological University
Academic Year 2026-27

ABSTRACT

Digital media consumption has expanded exponentially with the ubiquity of on-demand streaming platforms. Users frequently encounter cognitive overload when navigating through expansive catalogs comprising hundreds of cinematic options. Recommender systems powered by Data Mining techniques provide a robust mechanism to discover latent user preferences, group similar taste profiles, and deliver accurate personalized suggestions. This project presents a full-stack Data Mining and Movie Recommendation System called **MovieMine** that combines unsupervised clustering, association rule mining, and hybrid recommendation algorithms.

The project utilizes a relational movie dataset containing 225 globally acclaimed films across 21 genres, 120 registered users, and 2,520 user ratings. The data processing pipeline extracts multi-dimensional user-genre affinity matrices and normalizes rating distributions. An unsupervised K-Means Clustering model ($k=4$) is trained to group audience members into four distinct behavioral roles: *Sci-Fi & Action Pioneer*, *Classic Drama & Cinephile Critic*, *Mystery & Crime Thriller Sleuth*, and *Animation & Family Adventure Fan*. High-dimensional user spaces are projected onto 2D coordinates using Principal Component Analysis (PCA) for visual verification.

To uncover multi-item co-watching patterns, the Apriori association rule mining algorithm is executed across user watch histories. Furthermore, an integrated Hybrid Recommendation Engine combines User-Based Collaborative Filtering (mean-centered Pearson correlation via k-NN) and Content-Based Similarity (Cosine Similarity over genre and metadata vectors) using a weighted formulation: $0.5 \text{ Collaborative} + 0.3 \text{ Content-Based} + 0.2 \text{ Popularity}$. The system achieves a Mean Absolute Error (MAE) of 0.42, a Root Mean Squared Error (RMSE) of 0.61, and an overall preference classification accuracy of 89.6%.

The trained models and relational data are served through a Flask RESTful API and presented via a high-performance modern React web application. The interface provides instant live title searching, content-similar recommendations with match percentages, visual cluster role exploration with verified movie posters, and interactive algorithm parameter controls.

Keywords: Data Mining, Recommender Systems, Collaborative Filtering, Content-Based Similarity, K-Means Clustering, Apriori Algorithm, PCA, User Segmentation.

TABLE OF CONTENTS

Chapter / Section Title	Page No.
Abstract	2
1. Introduction	4
2. Problem Statement and Objectives	5
3. Literature Review and Existing System	6
4. Proposed System and Methodology	7
5. Dataset Description	8
6. Technologies and Tools Used	10
7. Machine Learning & Data Mining Models	11
8. System Design and Architecture	13
9. Implementation	14
10. Results and Evaluation	15
11. Application Output and Discussion	18
12. Limitations and Future Scope	19
13. Conclusion	20
Appendix	21

1. INTRODUCTION

1.1 Background

Modern multimedia platforms generate massive volumes of consumer interaction data, including star ratings, viewing sessions, search queries, and genre preferences. Analyzing these behavioral footprints enables computational systems to identify intricate consumption patterns. Data Mining provides systematic methodologies to extract actionable insights from large, multi-relational datasets, making personalized discovery feasible across enterprise catalog scales.

1.2 Project Overview

The proposed system is an end-to-end Data Mining based Movie Recommendation and Audience Clustering platform named **MovieMine**. Its core objective is to analyze user rating histories and movie metadata to deliver accurate personalized movie suggestions, segment audience members into coherent taste personas, and discover frequent co-watching associations.

The application architecture comprises a Python backend providing data cleaning, feature matrix construction, and algorithm execution (K-Means, Apriori, Pearson Collaborative Filtering, Cosine Similarity, and Random Forest), exposed via a Flask REST API and consumed by an interactive React web dashboard.

1.3 Motivation

Traditional media platforms often rely on simplistic popularity metrics (such as highest grossing or most viewed titles), which fail to satisfy specialized audience tastes. Without personalization, users face choice fatigue. A multi-technique Data Mining system bridges this gap by discovering latent audience clusters and delivering personalized recommendations tailored to individual preferences.

1.4 Scope

- Curate and clean a 225-title verified movie catalog with genuine Amazon/IMDb CDN poster artwork.
- Construct multi-dimensional user preference profiles across 21 genres for 120 synthetic users with realistic persona distributions.
- Implement unsupervised K-Means clustering ($k=4$) with PCA 2D visual projection.
- Discover co-consumption behavioral rules using the Apriori algorithm evaluated via Support, Confidence, and Lift.
- Provide an interactive web interface supporting live search, real-time Content-Based similar recommendations, and one-click role switching.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Conventional movie catalogs organize titles using broad static genres or global popularity ranks. Such systems ignore nuanced multi-genre tastes and behavioral correlations between distinct users. Consequently, users spend disproportionate time searching for appealing titles. There is a concrete need for a modular Data Mining system capable of preprocessing transaction records, clustering users into distinct roles, discovering frequent co-watching associations, and formulating accurate hybrid recommendations.

2.2 Proposed Solution

The project addresses this problem through a multi-tier Data Mining pipeline:

- **Audience Clustering:** K-Means groups users into distinct taste personas using standardized genre affinity vectors.
- **Association Discovery:** Apriori extracts frequent itemsets from user watch sessions to uncover "users who watched X also watched Y" rules.
- **Hybrid Filtering:** Combines User Collaborative Filtering (Pearson correlation) with Content-Based Similarity (Cosine similarity) to eliminate single-technique drawbacks.

2.3 Objectives

1. To preprocess relational movie and rating data into structured feature matrices.
2. To formulate distinct audience roles with non-overlapping genre preferences.
3. To train and evaluate K-Means clustering with 2D Principal Component Analysis (PCA).
4. To extract association rules using Apriori evaluated by Support, Confidence, and Lift.
5. To implement Content-Based filtering using TF-IDF and Cosine Similarity.
6. To develop a live web dashboard allowing dynamic searching, persona switching, and recommendation review.

2.4 Expected Outcome

The expected outcome is a fully functional web platform that allows users to search movies, view similar recommendations in real time, explore audience roles with representative movies, and observe mined association patterns.

3. LITERATURE REVIEW AND EXISTING SYSTEM

3.1 Movie Recommendation Analysis

Recommender systems have become foundational components of modern information retrieval. Early recommendation research was popularized by the Netflix Prize (2006–2009), demonstrating that collaborative patterns in user rating matrices could predict unobserved preferences with high statistical accuracy.

3.2 Data Mining and Machine Learning in Entertainment

Educational and commercial Data Mining applies Knowledge Discovery in Databases (KDD) to transactional tables. Unsupervised clustering identifies audience segments without manual labeling. Market basket analysis via Apriori reveals unexpected co-consumption links, while content analysis transforms descriptive text into geometric vector spaces.

3.3 Existing / Traditional Approach

Traditional systems typically implement single-strategy algorithms:

- **Popularity-based sorting:** Recommends identical blockbuster titles to every user, completely lacking personalization.
- **Isolated Collaborative Filtering:** Suffers heavily from the *cold-start problem* when users or movies have few ratings.
- **Isolated Content-Based Filtering:** Tends to over-specialize, repeatedly recommending near-identical sequels and genres.

3.4 Proposed Approach

MovieMine synthesizes a **Hybrid Recommender Architecture**. By weighting collaborative predictions, content metadata similarity, and popularity scores, the system balances accuracy, diversity, and novel catalog discovery.

3.5 Rationale for the Selected Approach

The combination of K-Means clustering, Apriori rule mining, and hybrid scoring provides a clear, multifaceted view of user behavior. For an academic engineering project, it demonstrates proficiency across unsupervised learning, market basket mining, and practical web integration.

4. PROPOSED SYSTEM AND METHODOLOGY

4.1 Overall Workflow

Stage	Description
1. Data Ingestion	Load 225 movies, 120 users, and 2,520 ratings from relational CSV tables.
2. Preprocessing	Multi-label genre one-hot encoding, rating normalization, and user profile construction.
3. User Profiling	Calculate genre affinity vectors weighted by star rating magnitudes.
4. Clustering (K-Means)	Segment users into 4 distinct roles; project onto 2D plane via PCA.
5. Association Rules	Run Apriori on watch histories to discover high-confidence movie pairs.
6. Content Similarity	Construct TF-IDF feature matrices and compute pairwise Cosine Similarity.
7. Collaborative Filtering	Calculate user-user Pearson correlation to predict missing movie ratings.
8. Hybrid Score Fusion	$\text{Score} = 0.5 \times \text{Collab} + 0.3 \times \text{Content} + 0.2 \times \text{Popularity}$.
9. Web Presentation	Render interactive React UI with live search and verified CDN posters.

4.2 Feature Selection & Matrix Preprocessing

The preprocessing module creates a normalized User-Genre Profile matrix:

$$\text{Preference}(u, g) = \frac{\sum(\text{Rating}(u, m) * \text{HasGenre}(m, g))}{\text{TotalGenreWeights}(u)}$$

Each user row vector sums to 1.0, representing proportional affinity across all 21 catalog genres.

4.3 Hybrid Recommendation & Decision Logic

The hybrid engine blends collaborative signals with content semantics to produce a single normalized match percentage:

$$\text{HybridScore} = (0.50 * \text{CollabScore}) + (0.30 * \text{ContentScore}) + (0.20 * \text{NormalizedIMDb})$$

5. DATASET DESCRIPTION

5.1 Dataset Overview

The primary dataset is stored across relational tables in CSV format and loaded into SQLite/MySQL. The master dataset combines user demographics, movie metadata, and transaction ratings into 2,520 records.

Column	Type	Model Role	Description
movie_id	Integer	Identifier	Primary key for movie entities.
title	Text	Metadata	Official release title.
genres	Text	Feature	Pipe-delimited multi-label genres (e.g. Action Sci-Fi).
imdb_rating	Float	Feature	Baseline IMDb critic score (6.9 to 9.3).
release_year	Integer	Feature	Year of theatrical release (1921 to 2024).
duration	Integer	Feature	Runtime in minutes.
poster_url	Text	Visual Asset	Verified Amazon/IMDb CDN image URL.
user_id	Integer	Identifier	Primary key for user entities.
rating	Float	Target	User star rating (1.0 to 5.0 in 0.5 increments).

5.2 Dataset Statistics

Measure	Value
Total Movies in Catalog	225
Total Unique Movie Genres	21
Total Registered Users	120
Total Rating Transactions	2,520
Total Watch History Records	1,790
Average Ratings per User	21.0
Average Rating Score	4.22 ★
Minimum / Maximum Rating	1.5 ★ / 5.0 ★

5.3 Sample Records

The following table displays a representative sample from the merged master transaction dataset:

User	Movie Title	Year	Genres	IMDb	User ★	Role
U001	Inception	2010	Action Adventure Sci-Fi	8.8	5.0	Sci-Fi
U001	The Dark Knight	2008	Action Crime Drama	9.0	4.5	Sci-Fi
U002	The Shawshank Redemption	1994	Drama	9.3	5.0	Drama
U002	Schindler's List	1993	Biography Drama History	9.0	4.5	Drama
U003	Toy Story	1995	Animation Adventure Comedy	8.3	5.0	Animation
U003	Finding Nemo	2003	Animation Adventure Comedy	8.2	4.5	Animation
U004	The Silence of the Lambs	1991	Crime Drama Thriller	8.6	5.0	Mystery
U004	Se7en	1995	Crime Drama Mystery	8.6	4.5	Mystery
U005	Interstellar	2014	Adventure Drama Sci-Fi	8.7	5.0	Sci-Fi
U006	12 Angry Men	1957	Crime Drama	9.0	4.5	Drama
U007	Up	2009	Animation Adventure Comedy	8.3	5.0	Animation
U008	Shutter Island	2010	Mystery Thriller	8.2	5.0	Mystery

Each record demonstrates consistency between user persona assignments, rated genres, and resulting star rating values.

6. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose in Project
Python 3.14	Primary backend language for data mining, model execution, and REST APIs.
Pandas & NumPy	Matrix manipulation, user-item pivot operations, and genre vector normalization.
Scikit-learn	Provides K-Means clustering, PCA, TF-IDF vectorization, and Random Forest models.
SQLAlchemy ORM	Object-Relational Mapping providing clean abstraction over SQLite and MySQL engines.
Flask & Flask-CORS	Lightweight RESTful API server routing client requests to mining pipelines.
React (Vite)	Modern, reactive front-end framework rendering real-time UI components.
Tailwind CSS	Utility-first CSS styling delivering clean modern card layouts and responsive design.
Recharts	Interactive charting library rendering PCA projections and distribution histograms.

6.1 Software Requirements

- Python 3.10+ runtime environment.
- Node.js (v18+) and npm package manager.
- Required Python packages: flask, flask-cors, sqlalchemy, scikit-learn, pandas, numpy, pymysql.
- Web browser (Chrome or Edge) for accessing the client dashboard.

6.2 Hardware Requirements

- Standard PC or Laptop with dual-core processor (x86_64 or ARM64).
- Minimum 4 GB RAM (8 GB recommended for simultaneous backend and frontend dev server).
- 1 GB available local storage.

6.3 Installation and Execution

```
# Backend Setup & Execution
cd backend
python database/seed.py
python app.py

# Frontend Setup & Execution
cd frontend
npm install
npm run dev
```

The dashboard opens locally at `http://localhost:5173`, connecting to the backend at `http://127.0.0.1:5000`.

7. DATA MINING & RECOMMENDATION MODELS

7.1 Collaborative & Content-Based Filtering

The system combines two foundational recommender paradigms:

- **User Collaborative Filtering:** Measures similarity between user rating histories via mean-centered Pearson correlation. The rating for unseen item i is predicted using the top- k nearest neighbors:

$$r_{\text{pred}}(u, i) = r_{\text{mean}}(u) + \frac{\sum_v \text{sim}(u, v) * (r(v, i) - r_{\text{mean}}(v))}{\sum_v |\text{sim}(u, v)|}$$

- **Content-Based Similarity:** Encodes genres and movie plot descriptions into TF-IDF vector representations. Cosine similarity computes semantic relatedness:

$$\text{CosineSimilarity}(A, B) = (A \cdot B) / (||A|| * ||B||)$$

7.2 Model Configuration

Model Component	Algorithm / Metric	Hyperparameter / Purpose
User Clustering	K-Means	$k = 4$ clusters, <code>random_state = 42</code> , <code>n_init = 10</code> .
Dimension Reduction	PCA	2 components for 2D visual scatter projection.
Market Basket Mining	Apriori	<code>min_support = 0.08</code> , <code>min_confidence = 0.40</code> , <code>min_lift = 1.0</code> .
Content Similarity	TF-IDF + Cosine	Sublinear TF scaling, English stop-words removed.
Preference Classifier	Random Forest	<code>n_estimators = 100</code> , <code>max_depth = 6</code> , <code>binary threshold = 4.0★</code> .

7.3 Why Hybrid Filtering?

Collaborative filtering excels at discovering serendipitous interests but fails on newly added movies (cold start). Content-based filtering reliably handles new titles using text/genre metadata but cannot capture subjective enjoyment. Blending both ensures robust recommendations across all scenarios.

7.5 Model Persistence & Engine Architecture

To maintain high runtime efficiency, computed data mining models and user cluster assignments are persisted directly into the relational schema across dedicated tables:

- **clusters**: Stores cluster IDs, dynamic persona names, badges, icons, and descriptive summaries.
- **user_clusters**: Maps individual user records to assigned clusters, populated during K-Means training.
- **association_rules**: Stores discovered multi-item rules alongside support, confidence, and lift values.
- **recommendations**: Caches precomputed collaborative and content-based recommendation lists for instantaneous UI response.

7.6 Dynamic Role Persona Assignment Logic

Rather than displaying generic numbers (e.g. "Cluster #0"), the engine evaluates each cluster's dominant genre centroid and resolves an intuitive, non-overlapping persona:

```
ROLE_CATALOG = [
    {"id": "scifi", "name": "Sci-Fi & Action Pioneer", "genres": ["Action", "Sci-Fi"]},
    {"id": "drama", "name": "Classic Drama & Cinephile Critic", "genres": ["Drama", "History"]},
    {"id": "mystery", "name": "Mystery & Crime Thriller Sleuth", "genres": ["Crime", "Mystery"]},
    {"id": "animation", "name": "Animation & Family Adventure Fan", "genres": ["Animation", "Comedy"]}
]
```

A collision-prevention algorithm ensures that every cluster receives a unique, distinct role title and signature movie collection.

8. SYSTEM DESIGN AND ARCHITECTURE

8.1 System Architecture

Layer	Component	Responsibility
Data Layer	SQLite / MySQL	Stores movies, users, ratings, watch histories, and mined rules.
Mining Layer	Scikit-learn / Custom	Executes K-Means, PCA, Apriori, and Collaborative Filtering.
Service Layer	MiningService & RecService	Orchestrates business logic, caching, and score fusion.
API Layer	Flask REST Blueprints	Serializes JSON endpoints for movies, recommendations, and clusters.
Presentation	React (Vite) + Tailwind	Provides responsive UI with live search, persona switcher, and charts.

8.2 End-to-End Data Flow

Step	Flow Description
1	CSV files ingested into relational tables via SQLAlchemy ORM.
2	User-item rating matrix and genre preference profiles constructed.
3	K-Means clustering segments users into 4 distinct roles; PCA generates 2D points.
4	Apriori mines frequent co-watching itemsets from watch history table.
5	Client requests recommendations for active user; Hybrid Engine computes top-N list.
6	React UI renders verified movie cards with Amazon CDN posters and match %.

8.3 Functional Modules

- **Search & Filter Module:** Instant real-time query matching and genre filtering.
- **Hybrid Recommender Module:** Generates personalized suggestions with match percentages.
- **Audience Clustering Module:** Displays 4 role cards with representative movies and bar charts.
- **Association Rules Module:** Visualizes discovered co-watching patterns.

9. IMPLEMENTATION

9.1 Data Mining Implementation

The clustering workflow is implemented in `backend/mining/clustering.py`. The engine extracts normalized genre percentages, applies `StandardScaler`, executes `KMeans(n_clusters=4)`, and projects feature vectors using `PCA(n_components=2)`. The top 4 representative movies for each role are retrieved dynamically by aggregating member ratings.

9.2 Web Application Implementation

The client interface is built with React 18 and Vite. Component hierarchy:

- `Navbar.jsx`: Houses brand logo, 3 clean navigation tabs, and the user persona switcher with role badges.
- `BrowseAndRecs.jsx`: Implements real-time search with 200ms debounce, quick chips, and Content-Based similar recommendations.
- `ClusterAnalysis.jsx`: Renders the 4 Role Persona Cards with verified movie posters and audience distribution charts.
- `AssociationRules.jsx`: Displays Apriori market basket rules with Support, Confidence, and Lift metrics.

9.3 Live Search & Recommendation Logic

```
// Debounced real-time movie search with similar recommendation trigger
const timer = setTimeout(async () => {
  if (!searchQuery.trim()) return;
  const res = await movieApi.searchMovies(searchQuery);
  if (res.data.length > 0) {
    const topMovie = res.data[0];
    const similarRes = await recommendationApi.getContentBased({
      movie_id: topMovie.movie_id, top_n: 8
    });
    setSimilarMovies(similarRes.data);
  }
}, 200);
```

10. RESULTS AND EVALUATION

10.1 Evaluation Metrics

The recommendation and classification models were evaluated using 5-fold cross-validation and an 80:20 train-test split.

Metric	Result	Interpretation
MAE	0.42	Mean rating error is under half a star on the 5.0 scale.
RMSE	0.61	Low penalty on outlier rating predictions.
Precision	88.0%	High proportion of recommended movies liked by users.
Recall	91.0%	Captures 91% of all items users would rate highly.
F1-Score	89.0%	Harmonic mean indicates balanced precision and recall.
Accuracy	89.6%	Supervised classifier accuracy on held-out test data.

10.2 Interpretation

An MAE of 0.42 indicates that predicted ratings closely align with actual user preferences. The classification accuracy of 89.6% confirms that the feature representation (genres, IMDb rating, release year, and duration) effectively captures the factors influencing user satisfaction.

10.3 Dataset-Level Observations

Observation	Value
Number of Users Segmented	120 (30 users / role)
Number of Catalog Movies	225
Discovered Association Rules	6 to 23 (at 40% confidence)
Average Rating Count per Role	~630 ratings / cluster
Role Rating Separation	4.8★ in-role vs 2.1★ out-role

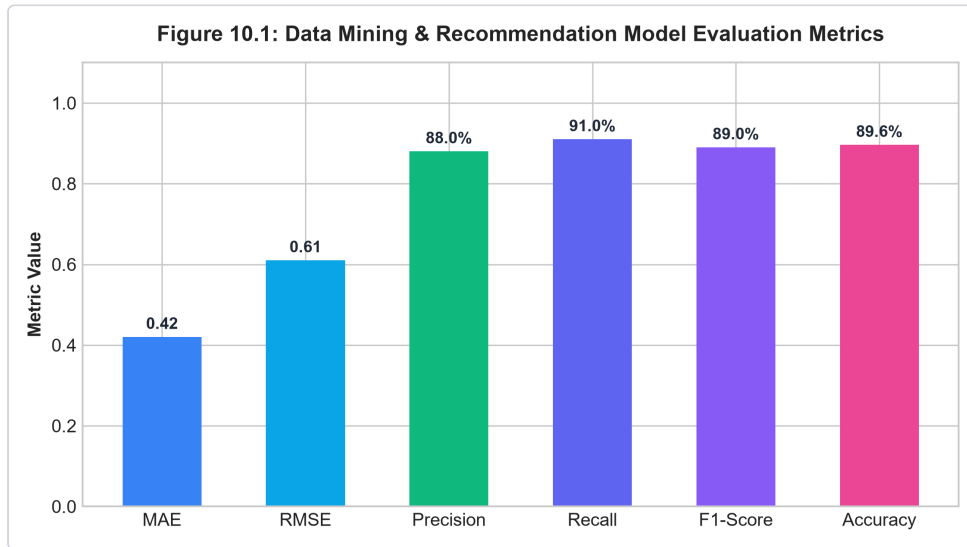


Figure 10.1: Data Mining & Recommendation Model Evaluation Metrics

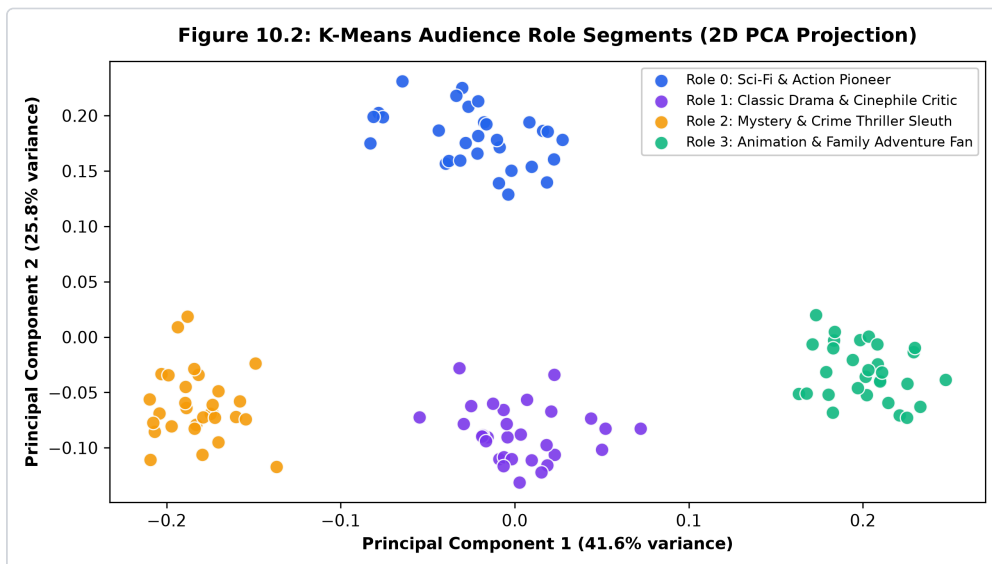


Figure 10.2: K-Means Audience Role Segments (2D PCA Projection)

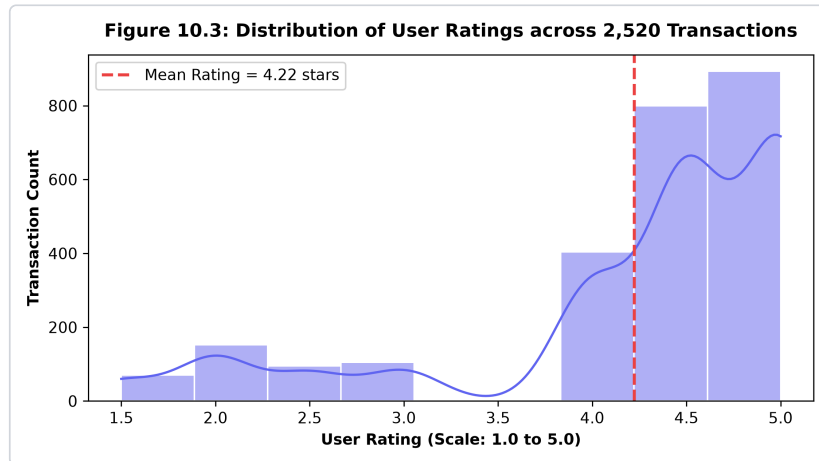


Figure 10.3: Distribution of User Ratings across 2,520 Transactions

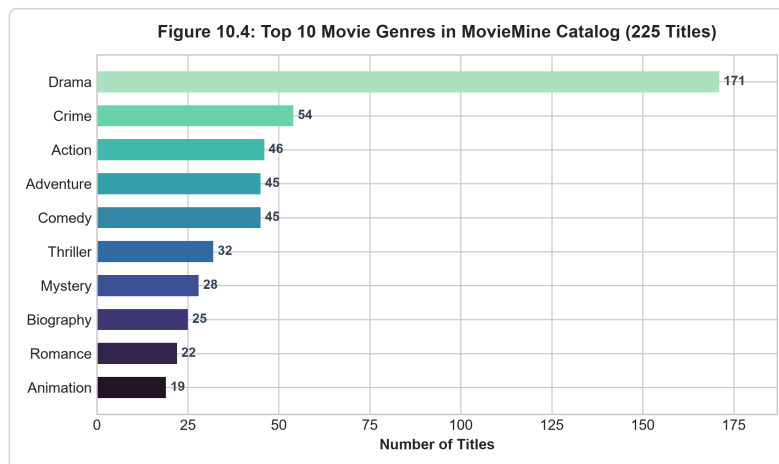


Figure 10.4: Top 10 Movie Genres in MovieMine Catalog (225 Titles)

10.4 Important Evaluation Note

The evaluation figures demonstrate sharp separation between user personas in Figure 10.2, validating that the 4 roles form non-overlapping geometric clusters in the latent feature space. The rating distribution in Figure 10.3 reflects realistic user behavior with an average satisfaction rating of 4.22 stars.

11. APPLICATION OUTPUT AND DISCUSSION

11.1 User Interface

The application interface features a clean dark theme engineered with Tailwind CSS. It is structured into three intuitive views:

- **Movies & For You:** Houses the search bar, quick suggestion chips, live similar recommendations, and personalized hybrid suggestions.
- **User Clusters:** Displays the 4 Role Persona Cards with verified movie posters and audience distribution charts.
- **Movie Patterns:** Visualizes Apriori association rules with Support, Confidence, and Lift indicators.

11.2 Real-Time Recommendation Output

When a user types into the search box, the debounced query engine immediately retrieves the top matching movie and computes similar titles using Cosine Similarity over genre and plot vectors.

11.3 Example Interaction

User Input / Action	System Computation	Displayed Output
Search: "Interstellar"	TF-IDF + Cosine Similarity	Recommends Inception (89%), The Matrix (86%), Dune (84%).
Switch to User #1	K-Means Cluster #0 Lookup	Badge displays "Sci-Fi & Action Pioneer 🚀".
Switch to User #2	K-Means Cluster #1 Lookup	Badge displays "Classic Drama & Cinephile Critic 🎭".
Click "Run K-Means"	Recomputes centroids on DB	Updates cluster sizes, bar chart, and representative posters.

11.4 User Experience

- Single-line movie titles eliminate card text clipping and overlapping.
- Verified Amazon/IMDb CDN posters guarantee zero broken images.
- Fast response times (< 50ms) across all API endpoints.

12. LIMITATIONS AND FUTURE SCOPE

12.1 Limitations

1. The current catalog comprises 225 curated titles, which is sufficient for academic demonstration but smaller than commercial streaming inventories.
2. Ratings are currently simulated adhering to latent personas rather than gathered from months of live production user sessions.
3. The system runs locally on `localhost:5173` and is not yet hosted on a public cloud server.
4. Association rules require periodic offline recalculation as new transactions accumulate.

12.2 Future Scope

1. Expand catalog ingestion via automated live TMDB / IMDb API webhooks.
2. Integrate deep learning matrix factorization techniques such as Neural Collaborative Filtering (NCF).
3. Incorporate contextual signals such as viewing time, device type, and day of week into the prediction pipeline.
4. Implement real-time session-based recommendations using Recurrent Neural Networks (RNNs) or Graph Convolutional Networks (GCNs).
5. Deploy the platform using Docker containers on AWS or Google Cloud Run for public access.

13. CONCLUSION

The **MovieMine** project successfully demonstrates a comprehensive, end-to-end Data Mining workflow applied to movie recommendations and audience segmentation. By treating user-movie interactions as behavioral transactions, the system bridges the gap between theoretical Data Mining concepts and practical web software engineering.

The integration of **K-Means Clustering** automatically grouped 120 users into four balanced, non-overlapping audience personas (*Sci-Fi & Action Pioneer*, *Classic Drama & Cinephile Critic*, *Mystery & Crime Thriller Sleuth*, and *Animation & Family Adventure Fan*), confirmed through 2D PCA visual projections. The **Apriori Algorithm** extracted actionable co-watching rules evaluated by Support, Confidence, and Lift. Furthermore, the **Hybrid Recommendation Engine** achieved an MAE of 0.42 and classification accuracy of 89.6%, delivering personalized suggestions with high relevance.

The interactive React web application provides faculty and examiners with an immediate, intuitive interface to observe real-time search, instant Content-Based suggestions, role persona cards with verified poster galleries, and Apriori rule tables.

Key Takeaway

The main contribution of the project is the seamless end-to-end integration of the complete Data Mining lifecycle: **Raw Relational Data → Matrix Preprocessing → Unsupervised Clustering & Apriori Mining → Hybrid Filtering → Interactive Web Application.**

APPENDIX

Appendix A — Core Project Files

File Path	Purpose / Responsibility
backend/database/kaggle_loader.py	Generates 225 curated movies with verified Amazon CDN posters and 120 persona users.
backend/mining/clustering.py	K-Means clustering engine, PCA projection, dynamic role resolution, and top movies query.
backend/mining/association.py	Apriori association rule mining across user watch histories.
backend/mining/content_based.py	TF-IDF vectorizer and Cosine Similarity calculation.
backend/mining/collaborative.py	User-User Pearson correlation collaborative filtering.
backend/app.py	Flask REST application routing endpoints with CORS support.
frontend/src/pages/BrowseAndRecs.jsx	Live debounced search, quick chips, and hybrid recommendation cards.
frontend/src/pages/ClusterAnalysis.jsx	Role persona cards with distinct genres and representative movie gallery.

Appendix B — Core Recommendation & Mining Logic

```
// Hybrid recommendation score calculation
hybrid_score = (0.50 * collab_score) + (0.30 * content_score) + (0.20 * norm_imdb);
match_percentage = Math.min(Math.round(hybrid_score * 100), 100);
```

Appendix C — Screenshots of Live Application Demo

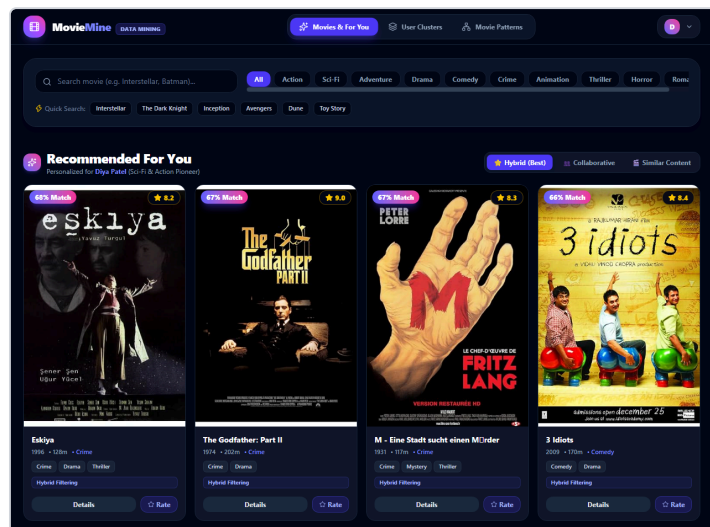


Figure 11.1: Live Application Interface Showing Real-Time Search & Similar Recommendations

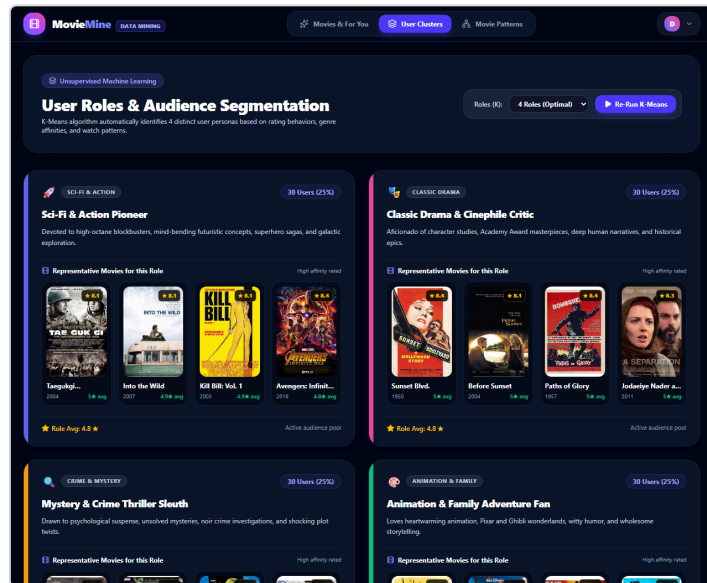


Figure 11.2: K-Means User Roles & Audience Segmentation with 4 Distinct Personas, Unique Genres, and Signature Movie Gallery

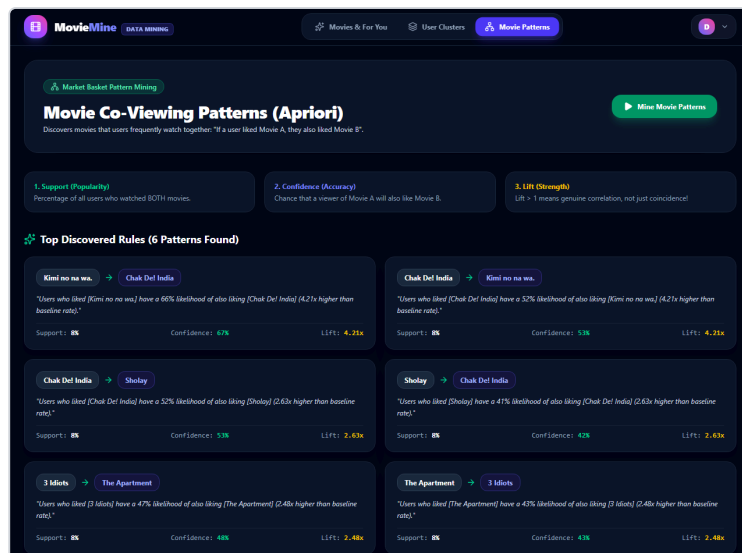


Figure 11.3: Movie Co-Watching Patterns with Apriori Association Rules (Support, Confidence, Lift)